

lab3

February 10, 2025

1 ETL with Bash

1.1 Objectives

The objective of this assignment is to implement an **ETL (Extract, Transform, Load) process using Bash scripting**. This exercise focuses on automating data extraction, transformation, and loading into a relational database using shell scripting techniques. The key learning goals are:

1. **Extracting Data:** Ingest and extract data from a remote source using Bash commands like `curl` or `wget`.
2. **Transforming Data:** Process and manipulate the extracted data using shell scripting tools (e.g., `cut`, `awk`, `sed`).
3. **Loading Data:** Import the transformed data into a **PostgreSQL database** using a Python script that creates a database table and loads the dataset.
4. **Scheduling the ETL Job:** Automate the ETL pipeline to run daily at a specific time using `cron` (job scheduler).

This assignment provides hands-on experience in **shell scripting, database interaction, data transformation, and job scheduling**, essential skills for data engineering roles.

1.2 Approach and Methodology

The ETL pipeline is implemented using a structured approach that follows these key steps:

1. **Extract Phase:**
 - Use Bash commands (`curl` or `wget`) to download the dataset from a remote URL.
 - Implement error handling to ensure the dataset is successfully retrieved before proceeding.
2. **Transform Phase:**
 - Process the extracted dataset using tools like `cut`, `awk`, and `sed` to select specific columns and clean the data.
 - Ensure consistency and format the data appropriately for database insertion.
3. **Load Phase:**
 - Execute a Python script to connect to a PostgreSQL database and insert the transformed data.
 - The script will create the necessary table (if it does not exist) and load the cleaned dataset.
4. **Automation with Cron:**
 - Schedule the Bash script to run at a fixed time daily using `crontab`.
 - Ensure logs are generated for tracking execution success or failures.

This methodology ensures that the ETL process runs efficiently and can be easily maintained or modified for different datasets.

1.3 Expected Outcomes

Upon completing this assignment, the following outcomes are anticipated:

- A fully functional **ETL pipeline** that automates data extraction, transformation, and loading into a PostgreSQL database.
- A **Bash script** that efficiently processes and cleans the data.
- A **Python script** that seamlessly interacts with the database to insert transformed data.
- A scheduled job using **cron**, ensuring the ETL process runs daily without manual intervention.
- Improved understanding of **shell scripting, automation, and data processing techniques**.
- Experience handling **real-world datasets and integrating Bash with database systems**.

This assignment will help build **practical data engineering skills** applicable to real-world ETL workflows.

1.4 Task 1: Creating a Bash Script File and Setting Permissions

1.4.1 Step 1: Create the Bash Script File

To begin the ETL process, we create a Bash script file named `ETL_script.sh`:

```
touch ETL_script.sh
```

This command creates an empty file that will store the Bash commands for the ETL pipeline.

1.4.2 Step 2: Check File Permissions

To verify the default permissions of the file, use the `ls -l` command:

```
ls -l ETL_script.sh
```

Output:

```
-rw-rw-r-- 1 simpcy simpcy 0 Feb 10 13:10 ETL_script.sh
```

Explanation: - rw- → Owner has **read and write** permissions. - rw- → Group members have **read and write** permission. - r-- → Others have **read-only** permission. - The **execute (x)** permission is missing, meaning the script cannot be executed yet.

1.4.3 Step 3: Grant Execute Permission

To allow the script to be executed, update its permissions using `chmod`:

```
chmod +x ETL_script.sh
```

This command adds **execute (x) permission** for the owner, making the script executable.

1.4.4 Step 4: Verify Updated Permissions

Re-run the `ls -l` command to confirm the change:

```
ls -l ETL_script.sh
```

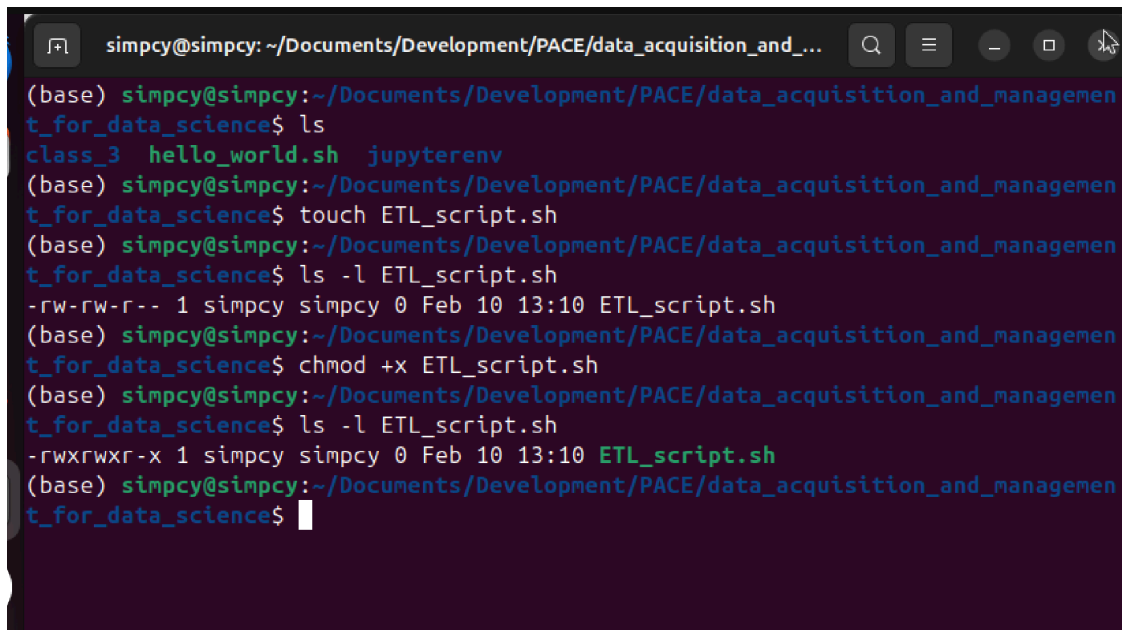
Expected output:

```
-rwxrwxr-x 1 simpcy simpcy 0 Feb 10 13:10 ETL_script.sh
```

Explanation: - rwx → Owner can **read, write, and execute**. - rwx → Group members can **read, write, and execute***. - r-x → Others can **read and execute**.

```
[74]: from IPython.display import display, Image

display(Image(filename="screenshots/create_file.png"))
```



```
simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_...
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_managemen
t_for_data_science$ ls
class_3  hello_world.sh  jupyterenv
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_managemen
t_for_data_science$ touch ETL_script.sh
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_managemen
t_for_data_science$ ls -l ETL_script.sh
-rw-rw-r-- 1 simpcy simpcy 0 Feb 10 13:10 ETL_script.sh
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_managemen
t_for_data_science$ chmod +x ETL_script.sh
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_managemen
t_for_data_science$ ls -l ETL_script.sh
-rwxrwxr-x 1 simpcy simpcy 0 Feb 10 13:10 ETL_script.sh
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_managemen
t_for_data_science$
```

1.5 Task 2: Opening the Script in a Text Editor

1.5.1 Step 1: Open the Script in Nano

The easiest way to edit the script is using nano:

```
nano ETL_script.sh
```

- This will open the file inside the **Nano text editor**.

Alternative Editors: - **Vim:** `vim ETL_script.sh` - **VS Code:** `code ETL_script.sh`

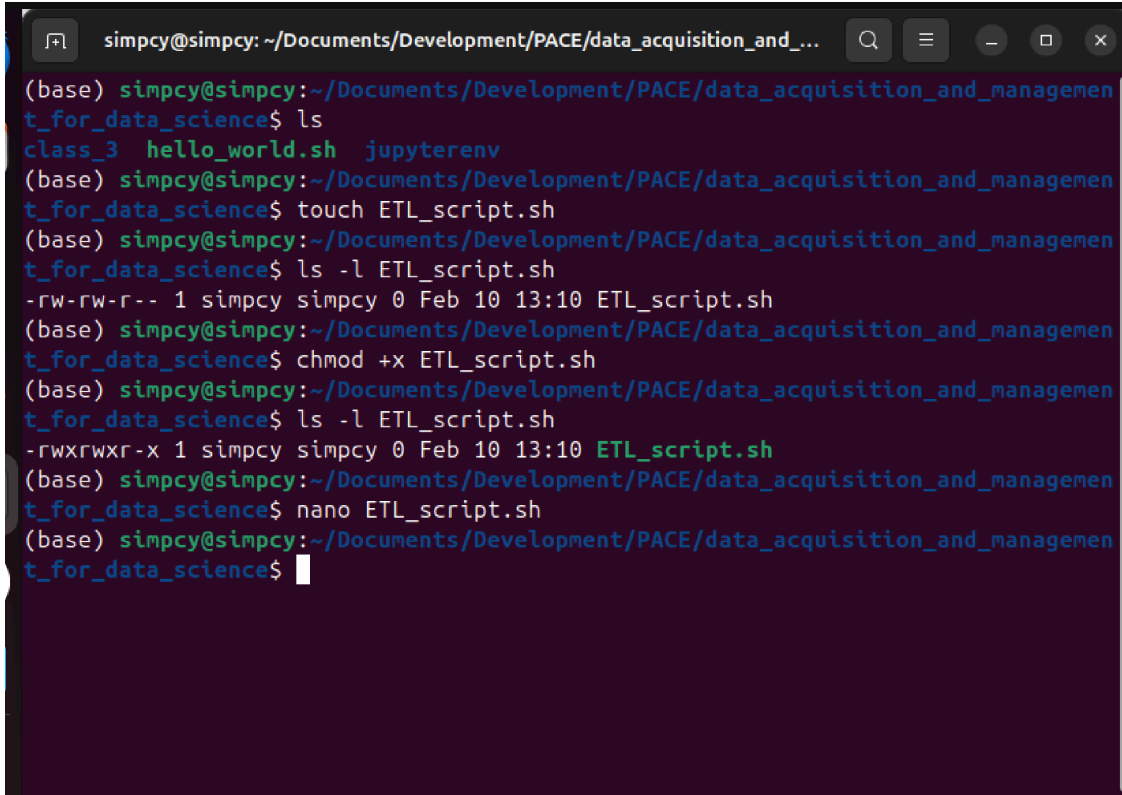
1.5.2 Step 2: Confirm the File is Opened

Once the command is executed, the terminal should display an **empty editor** where the script can be written.

To **exit nano**: - **CTRL + X** to exit. - If prompted to save changes, I'd press **Y**, then **Enter**.

```
[75]: # open file in nano command nano ETL_script.sh
```

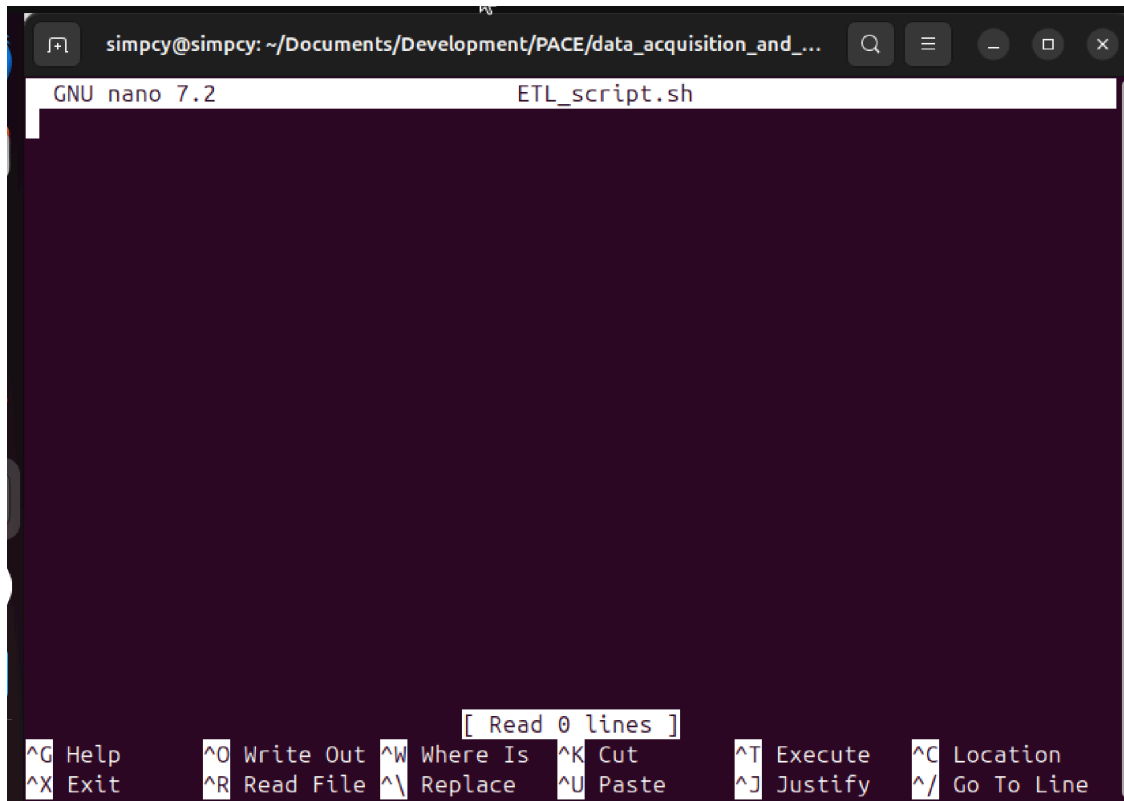
```
display(Image(filename="screenshots/nano_open_command.png"))
```

A terminal window screenshot showing a series of commands and their outputs. The user is in a directory ~/Documents/Development/PACE/data_acquisition_and_management_for_data_science. The commands executed are: 'ls' (listing files: class_3, hello_world.sh, jupyterenv), 'touch ETL_script.sh' (creating the file), 'ls -l ETL_script.sh' (showing permissions -rw-rw-r--), 'chmod +x ETL_script.sh' (making it executable), 'ls -l ETL_script.sh' (showing permissions -rwxrwxr-x), and 'nano ETL_script.sh' (opening the file in the nano editor). The terminal prompt is (base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science\$.

```
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ ls
class_3  hello_world.sh  jupyterenv
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ touch ETL_script.sh
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ ls -l ETL_script.sh
-rw-rw-r-- 1 simpcy simpcy 0 Feb 10 13:10 ETL_script.sh
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ chmod +x ETL_script.sh
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ ls -l ETL_script.sh
-rwxrwxr-x 1 simpcy simpcy 0 Feb 10 13:10 ETL_script.sh
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ nano ETL_script.sh
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$
```

```
[76]: # opened file in nano file nano
```

```
display(Image(filename="screenshots/open_nano_env.png"))
```



1.6 Task 3: Initializing the Bash Script and Defining Variables

1.6.1 Step 1: Add the Shebang (#!)

The **shebang** (**#!**) specifies the interpreter that will execute the script. Add the following line at the top of the file:

```
#!/bin/bash
```

This ensures that the script runs using the **Bash shell**.

1.6.2 Step 2: Define Variables

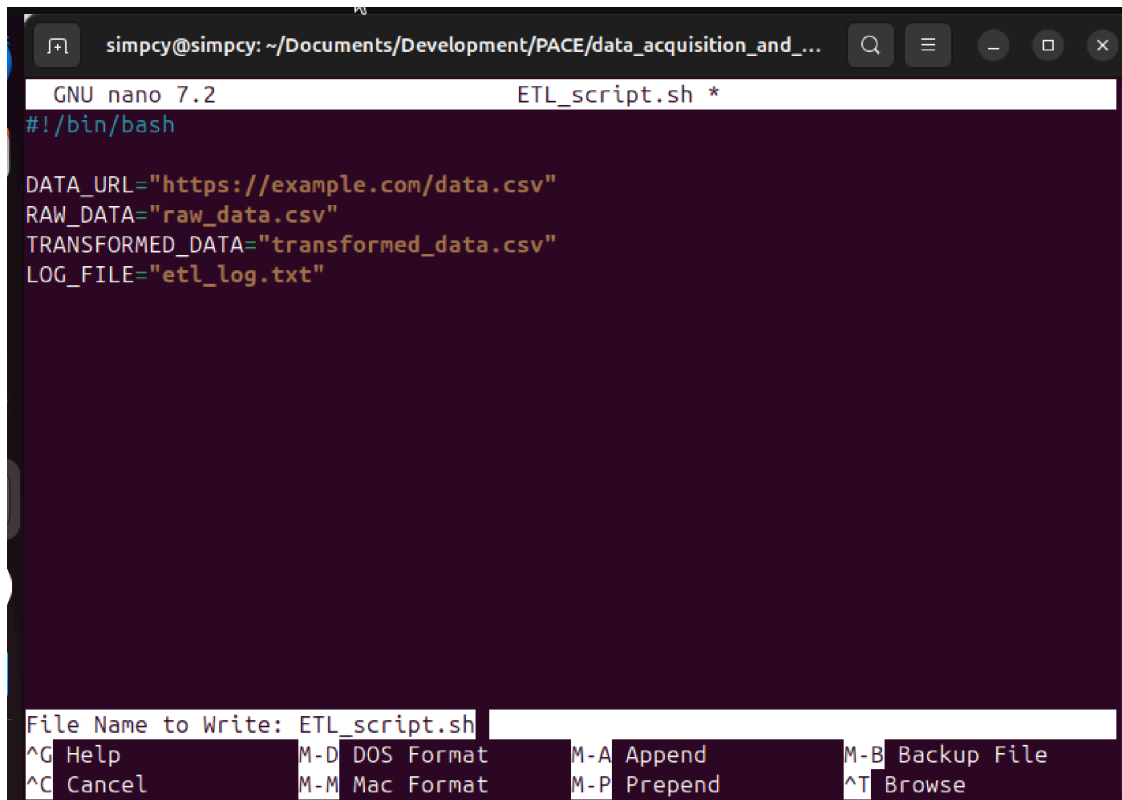
To maintain clarity and efficiency, define key variables at the beginning of the script:

```
# Define variables for dataset URL, filenames, and log file  
DATA_URL="https://example.com/data.csv"  
RAW_DATA="raw_data.csv"  
TRANSFORMED_DATA="transformed_data.csv"  
LOG_FILE="etl_log.txt"
```

- DATA_URL → Stores the URL of the dataset to be downloaded.
- RAW_DATA → Specifies the name of the file to store extracted data.
- TRANSFORMED_DATA → Stores the filename of the cleaned, processed data.
- LOG_FILE → Keeps track of ETL execution logs.

[77]: *# editing file in nano*

```
display(Image(filename="screenshots/write_out_step.png"))
```



```
simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_...
GNU nano 7.2 ETL_script.sh *
#!/bin/bash

DATA_URL="https://example.com/data.csv"
RAW_DATA="raw_data.csv"
TRANSFORMED_DATA="transformed_data.csv"
LOG_FILE="etl_log.txt"

File Name to Write: ETL_script.sh
^G Help      M-D DOS Format  M-A Append    M-B Backup File
^C Cancel    M-M Mac Format  M-P Prepend   ^T Browse
```

1.7 Task 4: Automating Data Extraction in the Script

1.7.1 Step 1: Implement Extraction in the Script

To automate the extraction process, add the following lines inside `ETL_script.sh`: To extract the dataset, use the `curl` or `wget` command to download the COVID-19 data from the provided URL:

Extract Phase: Download dataset

```
DATA_URL="https://raw.githubusercontent.com/owid/covid-19-data/master/public/da>
```

```
RAW_DATA="raw_owid_covid_data.csv"
```

```
LOG_FILE="ETL_log.txt"
```

```
echo "$(date): Starting data extraction..." >> $LOG_FILE
```

```
curl -o $RAW_DATA $DATA_URL
```

```
# Verify if the file was downloaded
```

```
if [[ -f "$RAW_DATA" ]]; then
```

```
    echo "$(date): Data extraction successful" >> $LOG_FILE
```

```
else
```

```
    echo "${date}: Data extraction failed" >> $LOG_FILE
fi
```

This command fetches the dataset and saves it as `raw_owid_covid_data.csv` in the current directory.

1.7.2 Step 2: Save and Close the File

Once the extraction script is written, save and close the file in Nano: - **Press CTRL + O**, then **Enter** to save the changes. - **Press CTRL + X** to exit.

1.7.3 Step 3: Run the ETL Script

```
./ETL_script.sh
```

1.7.4 What Happens?

1. The script **download** the COVID-19 dataset.
2. It **logs** the extraction process into `ETL_log.txt`.
3. I verify if the file exists by running:

```
ls -l raw_owid_covid_data.csv
```

1.7.5 Output

```
-rw-rw-r-- 1 simpcy simpcy 98391483 Feb 10 14:49 raw_owid_covid_data.csv
```

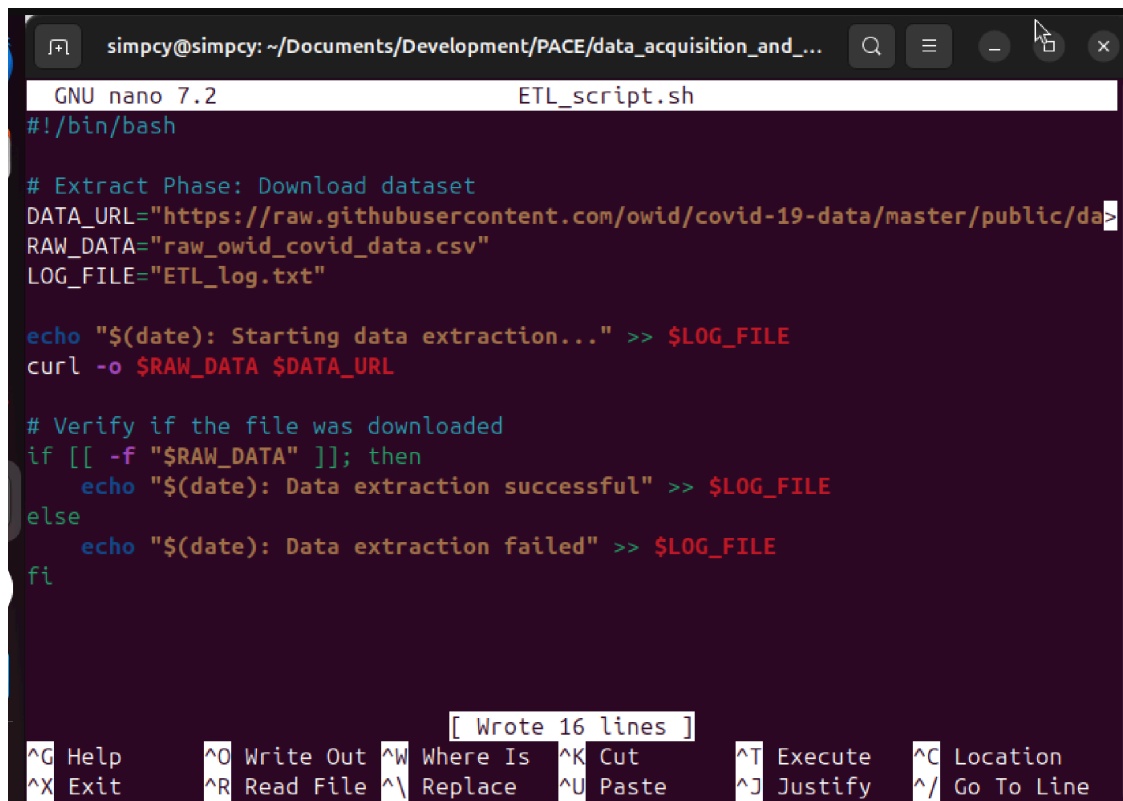
And the log file:

```
Mon Feb 10 02:49:40 PM UTC 2025: Starting data extraction...
```

```
Mon Feb 10 02:49:44 PM UTC 2025: Data extraction successful
```

[78]: *#download script*

```
display(Image(filename="screenshots/download_raw_data_script.png"))
```



```
simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_...
GNU nano 7.2 ETL_script.sh
#!/bin/bash

# Extract Phase: Download dataset
DATA_URL="https://raw.githubusercontent.com/owid/covid-19-data/master/public/data"
RAW_DATA="raw_owid_covid_data.csv"
LOG_FILE="ETL_log.txt"

echo "$(date): Starting data extraction..." >> $LOG_FILE
curl -o $RAW_DATA $DATA_URL

# Verify if the file was downloaded
if [[ -f "$RAW_DATA" ]]; then
    echo "$(date): Data extraction successful" >> $LOG_FILE
else
    echo "$(date): Data extraction failed" >> $LOG_FILE
fi

[ Wrote 16 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

```
[79]: # execute script and verify data is downloaded

display(Image(filename="screenshots/data_downloaded.png"))
```



```
# view last 10 lines of csv
display(Image(filename="screenshots/tail_raw_data.png"))
```

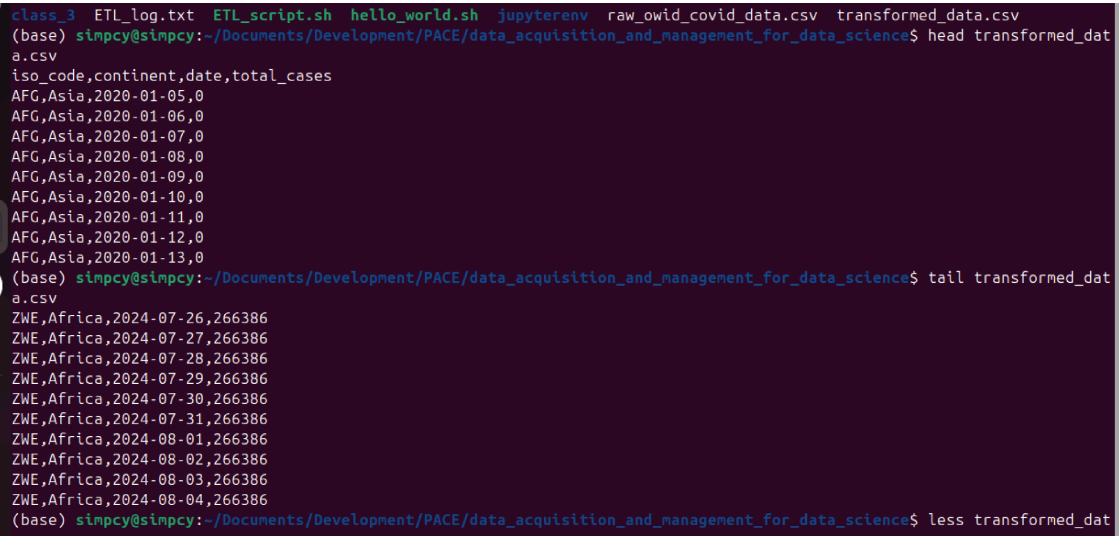
[illegible]

```
# view csv interactively
display(Image(filename="screenshots/less_raw_data.png"))
```



```
[83]: # show transformed csv
```

```
display(Image(filename="screenshots/transformed_csv.png"))
```



```
class_3 ETL_log.txt ETL_script.sh hello_world.sh jupyterenv raw_owid_covid_data.csv transformed_data.csv
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ head transformed_data.csv
iso_code,continent,date,total_cases
AFG,Asia,2020-01-05,0
AFG,Asia,2020-01-06,0
AFG,Asia,2020-01-07,0
AFG,Asia,2020-01-08,0
AFG,Asia,2020-01-09,0
AFG,Asia,2020-01-10,0
AFG,Asia,2020-01-11,0
AFG,Asia,2020-01-12,0
AFG,Asia,2020-01-13,0
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ tail transformed_data.csv
ZWE,Africa,2024-07-26,266386
ZWE,Africa,2024-07-27,266386
ZWE,Africa,2024-07-28,266386
ZWE,Africa,2024-07-29,266386
ZWE,Africa,2024-07-30,266386
ZWE,Africa,2024-07-31,266386
ZWE,Africa,2024-08-01,266386
ZWE,Africa,2024-08-02,266386
ZWE,Africa,2024-08-03,266386
ZWE,Africa,2024-08-04,266386
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ less transformed_data.csv
```

1.9 Task 6: Loading Data into PostgreSQL

1.9.1 Step 1: Create a Python Script for Data Loading

To load the transformed data into a PostgreSQL database, we use a Python script that: 1. Connects to a PostgreSQL instance. 2. Creates a database (covid_db) and a table (COVID) if they do not exist. 3. Loads the transformed data using the COPY command.

Created a new Python script named `load_to_postgres.py` and add the following code:

```
import psycopg2
import os
import sys

# Define database connection parameters
db_name = os.getenv('DB_NAME', 'covid_db')
db_user = os.getenv('DB_USER', 'username')
db_password = os.getenv('DB_PASSWORD', 'password')
db_host = os.getenv('DB_HOST', 'localhost')
db_port = os.getenv('DB_PORT', '5432')

# Define input file
transformed_data = "transformed_data.csv"

conn = None
cursor = None
```

```

try:
    # Connect to PostgreSQL default database to check/create covid_db
    temp_conn = psycopg2.connect(
        dbname="postgres", user=db_user, password=db_password, host=db_host, port=db_port
    )
    temp_conn.autocommit = True
    temp_cursor = temp_conn.cursor()

    # Create the database if it doesn't exist
    temp_cursor.execute(f"SELECT 1 FROM pg_database WHERE datname = '{db_name}'")
    if not temp_cursor.fetchone():
        temp_cursor.execute(f"CREATE DATABASE {db_name}")

    temp_cursor.close()
    temp_conn.close()

    # Now, connect to the actual database
    conn = psycopg2.connect(
        dbname=db_name, user=db_user, password=db_password, host=db_host, port=db_port
    )
    cursor = conn.cursor()

    # Create table if it doesn't exist
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS COVID (
            iso_code VARCHAR(10),
            continent VARCHAR(50),
            date DATE,
            total_cases BIGINT
        );
    """)

    # Load data into the table
    with open(transformed_data, 'r') as file:
        cursor.copy_expert("COPY COVID FROM STDIN WITH CSV HEADER", file)

    conn.commit()
    print("Data successfully loaded into PostgreSQL.")

except psycopg2.OperationalError as e:
    print("Database connection error:", e)
    sys.exit(1)
except Exception as e:
    print("Error loading data:", e)
    sys.exit(1)
finally:
    if cursor:
        cursor.close()

```

```

if conn:
    conn.close()

```

1.9.2 Step 2: Modify ETL Script to Call Python Loader

Update ETL_script.sh to include the database loading step:

```

# Load Phase: Load transformed data into PostgreSQL
echo "$(date): Starting data load into PostgreSQL..." >> $LOG_FILE
python3 load_to_postgres.py

if [[ $? -eq 0 ]]; then
    echo "$(date): Data successfully loaded into PostgreSQL" >> $LOG_FILE
else
    echo "$(date): Data load failed" >> $LOG_FILE
fi

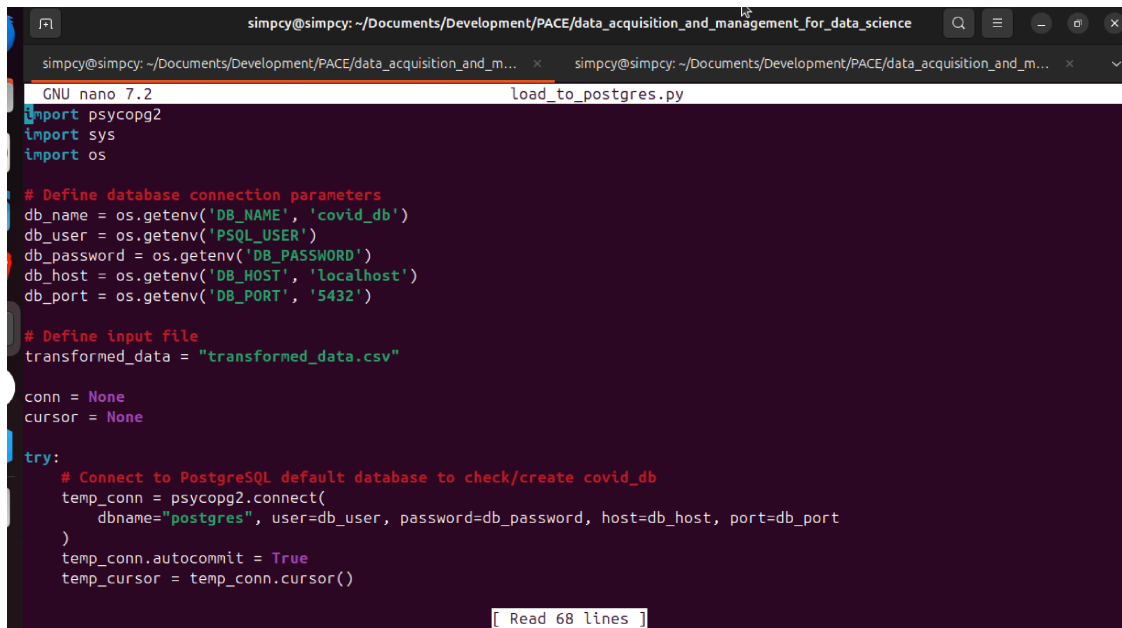
```

[84]: *# python script file*

```

display(Image(filename="screenshots/py_script1.png"))
display(Image(filename="screenshots/py_script2.png"))
display(Image(filename="screenshots/py_script3.png"))

```



```

simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_management_for_data_science
simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_m... x  simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_m... x v
GNU nano 7.2 load_to_postgres.py
import psycopg2
import sys
import os

# Define database connection parameters
db_name = os.getenv('DB_NAME', 'covid_db')
db_user = os.getenv('PSQL_USER')
db_password = os.getenv('DB_PASSWORD')
db_host = os.getenv('DB_HOST', 'localhost')
db_port = os.getenv('DB_PORT', '5432')

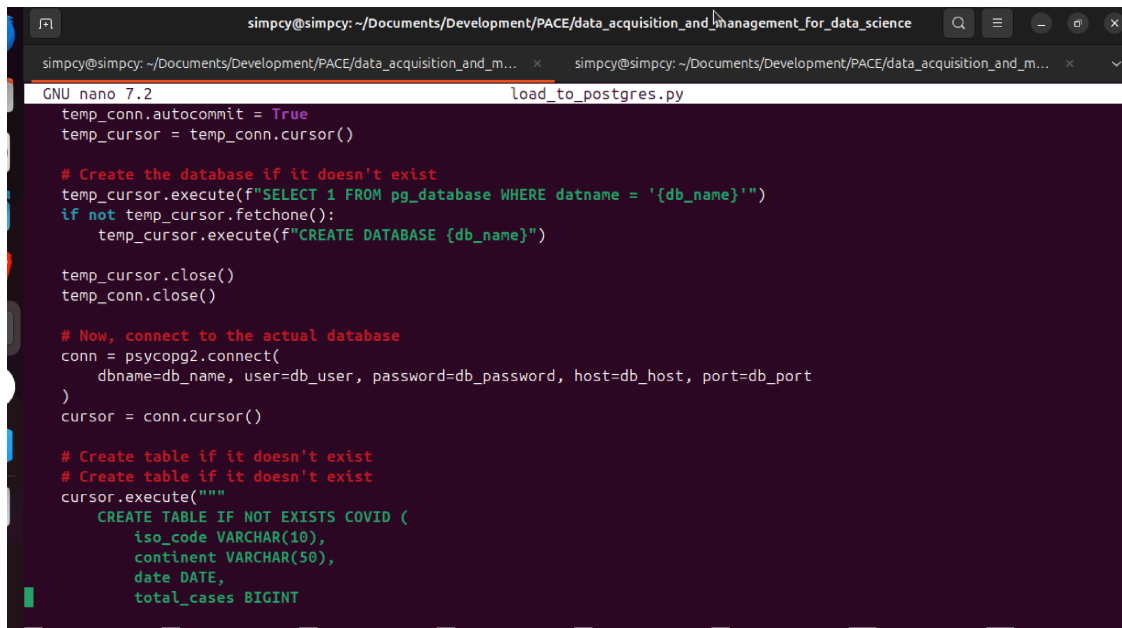
# Define input file
transformed_data = "transformed_data.csv"

conn = None
cursor = None

try:
    # Connect to PostgreSQL default database to check/create covid_db
    temp_conn = psycopg2.connect(
        dbname="postgres", user=db_user, password=db_password, host=db_host, port=db_port
    )
    temp_conn.autocommit = True
    temp_cursor = temp_conn.cursor()

```

[Read 68 lines]



```
simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_management_for_data_science
GNU nano 7.2 load_to_postgres.py
temp_conn.autocommit = True
temp_cursor = temp_conn.cursor()

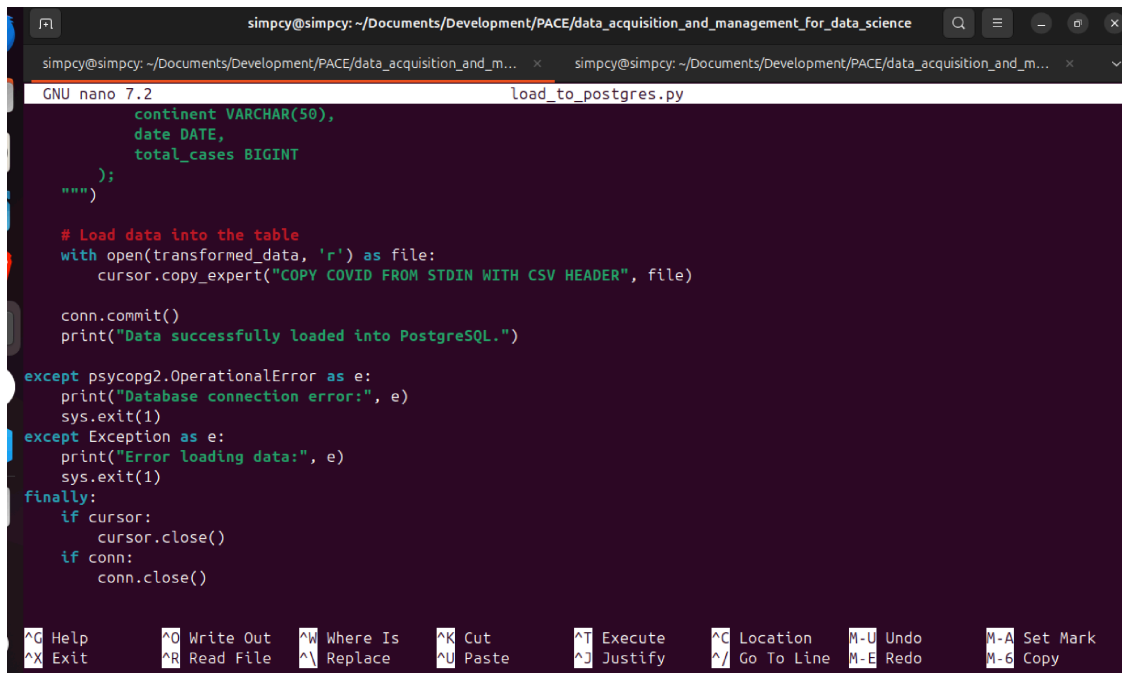
# Create the database if it doesn't exist
temp_cursor.execute(f"SELECT 1 FROM pg_database WHERE datname = '{db_name}'")
if not temp_cursor.fetchone():
    temp_cursor.execute(f"CREATE DATABASE {db_name}")

temp_cursor.close()
temp_conn.close()

# Now, connect to the actual database
conn = psycopg2.connect(
    dbname=db_name, user=db_user, password=db_password, host=db_host, port=db_port
)
cursor = conn.cursor()

# Create table if it doesn't exist
# Create table if it doesn't exist
cursor.execute("""
    CREATE TABLE IF NOT EXISTS COVID (
        iso_code VARCHAR(10),
        continent VARCHAR(50),
        date DATE,
        total_cases BIGINT

```



```

        continent VARCHAR(50),
        date DATE,
        total_cases BIGINT
    );
""")

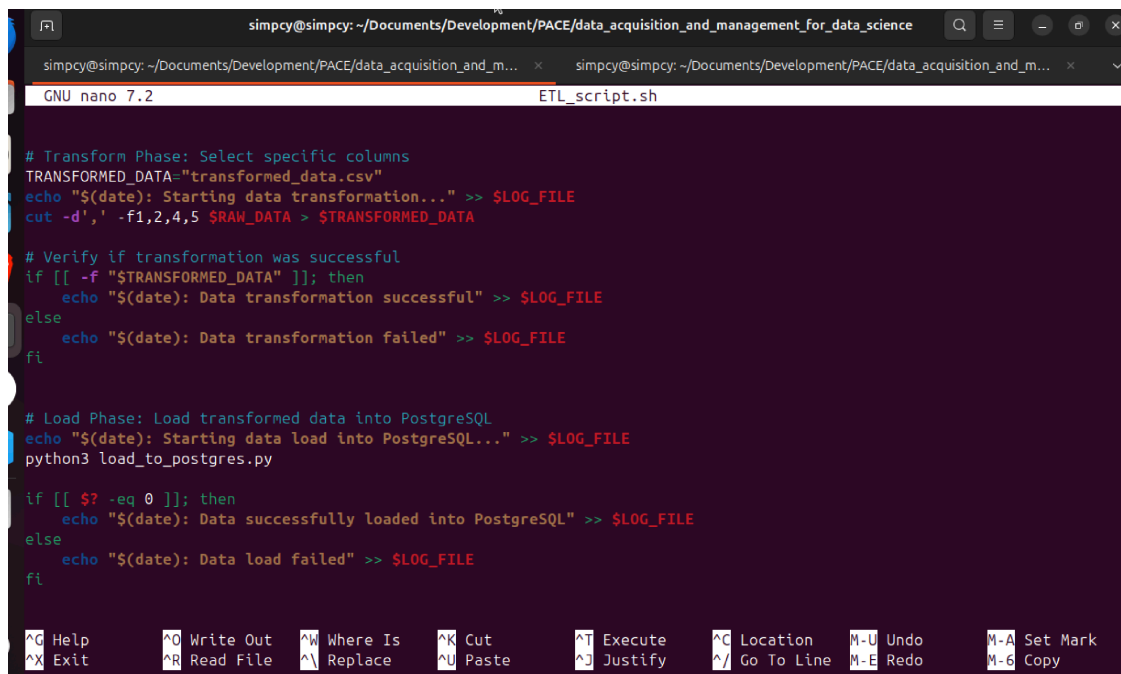
# Load data into the table
with open(transformed_data, 'r') as file:
    cursor.copy_expert("COPY COVID FROM STDIN WITH CSV HEADER", file)

conn.commit()
print("Data successfully loaded into PostgreSQL.")

except psycopg2.OperationalError as e:
    print("Database connection error:", e)
    sys.exit(1)
except Exception as e:
    print("Error loading data:", e)
    sys.exit(1)
finally:
    if cursor:
        cursor.close()
    if conn:
        conn.close()

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-6 Copy
```

```
[85]: # updated script
display(Image(filename="screenshots/script_run_py.png"))
```

```
simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_management_for_data_science
simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_m...  simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_m...
GNU nano 7.2 ETL_script.sh

# Transform Phase: Select specific columns
TRANSFORMED_DATA="transformed_data.csv"
echo "$(date): Starting data transformation..." >> $LOG_FILE
cut -d',' -f1,2,4,5 $RAW_DATA > $TRANSFORMED_DATA

# Verify if transformation was successful
if [[ -f "$TRANSFORMED_DATA" ]]; then
    echo "$(date): Data transformation successful" >> $LOG_FILE
else
    echo "$(date): Data transformation failed" >> $LOG_FILE
fi

# Load Phase: Load transformed data into PostgreSQL
echo "$(date): Starting data load into PostgreSQL..." >> $LOG_FILE
python3 load_to_postgres.py

if [[ $? -eq 0 ]]; then
    echo "$(date): Data successfully loaded into PostgreSQL" >> $LOG_FILE
else
    echo "$(date): Data load failed" >> $LOG_FILE
fi

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo      M-6 Copy
```

1.10 Task 7: Running the ETL Process and Logging

1.10.1 Step 1: Execute the Bash Script

To run the entire ETL process, execute the script:

```
./ETL_script.sh
```

This command will trigger the **Extract, Transform, and Load** phases sequentially.

1.10.2 Step 2: Monitor the Execution Logs

The script logs each stage in `ETL_log.txt`. To check the logs, run:

```
cat etl_log.txt
```

2. Check if Data was Transformed Correctly:

```
head transformed_data.csv
```

Expected Output:

```
iso_code,continent,date,total_cases
AFG,Asia,2020-01-05,0
AFG,Asia,2020-01-06,0
```

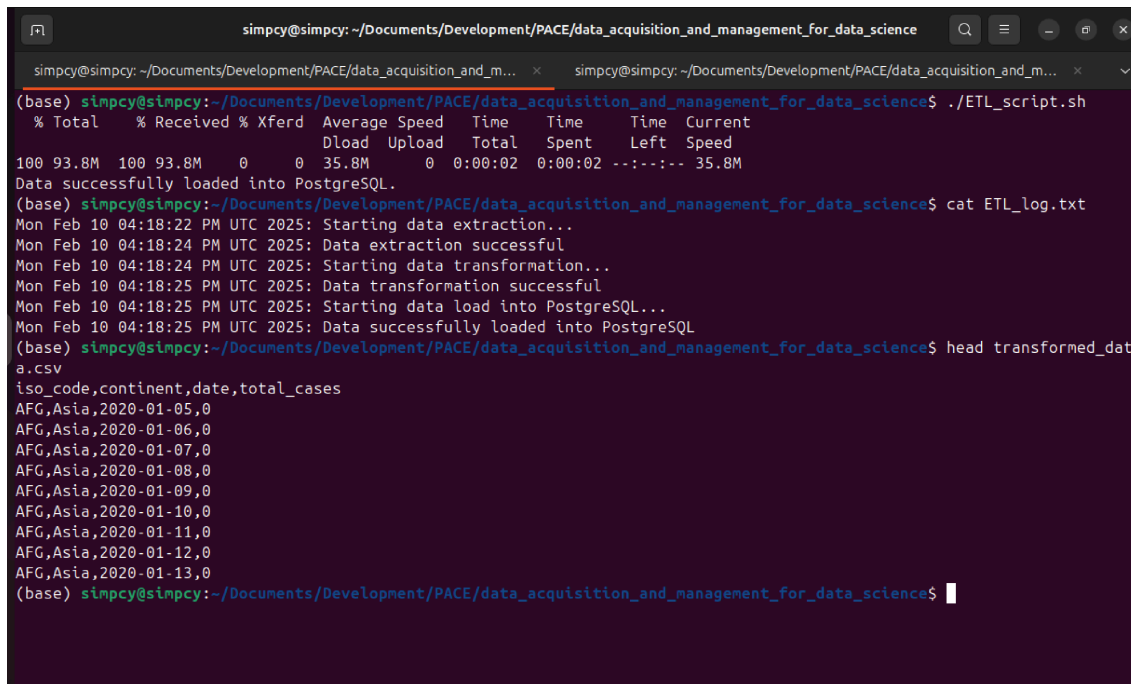
1.10.3 Step 4: Verify the Data Load

Run the following SQL query in PostgreSQL to check if data is loaded successfully:

```
SELECT * FROM COVID LIMIT 10;
```


[86]: *# run entire script*

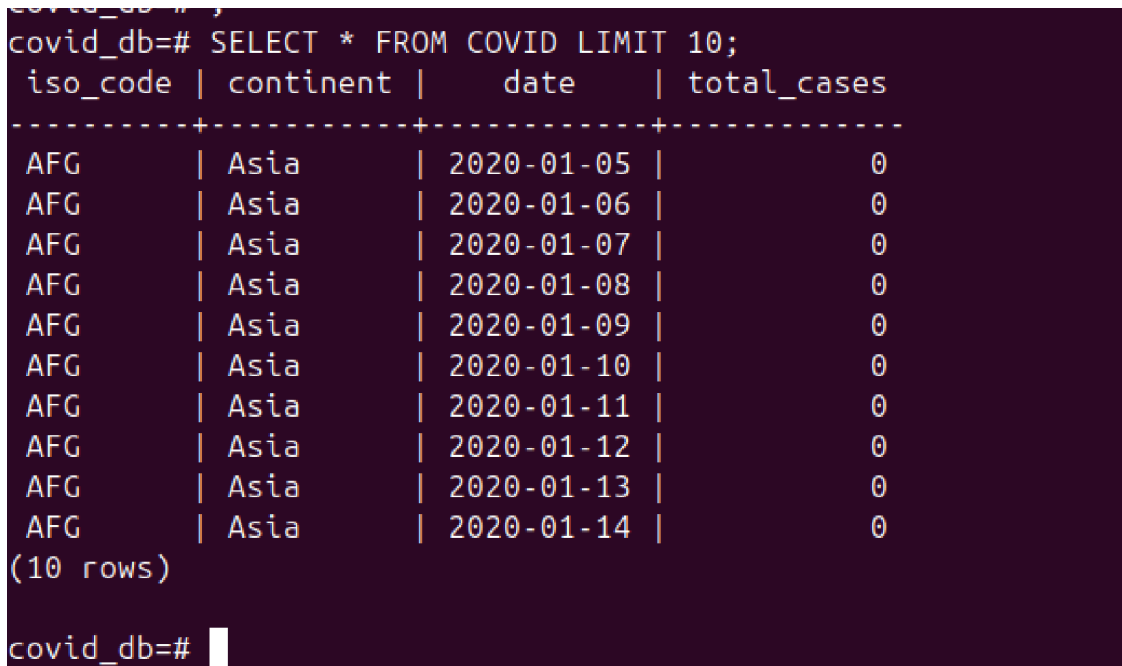
```
display(Image(filename="screenshots/run_bash_script.png"))
```



```
simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_management_for_data_science
simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_m... x  simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_m... x
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ ./ETL_script.sh
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
   Dload  Upload   Total   Spent    Left   Speed
100 93.8M  100 93.8M    0     0  35.8M      0  0:00:02  0:00:02 --:--:-- 35.8M
Data successfully loaded into PostgreSQL.
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ cat ETL_log.txt
Mon Feb 10 04:18:22 PM UTC 2025: Starting data extraction...
Mon Feb 10 04:18:24 PM UTC 2025: Data extraction successful
Mon Feb 10 04:18:24 PM UTC 2025: Starting data transformation...
Mon Feb 10 04:18:25 PM UTC 2025: Data transformation successful
Mon Feb 10 04:18:25 PM UTC 2025: Starting data load into PostgreSQL...
Mon Feb 10 04:18:25 PM UTC 2025: Data successfully loaded into PostgreSQL
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ head transformed_data.csv
iso_code,continent,date,total_cases
AFG,Asia,2020-01-05,0
AFG,Asia,2020-01-06,0
AFG,Asia,2020-01-07,0
AFG,Asia,2020-01-08,0
AFG,Asia,2020-01-09,0
AFG,Asia,2020-01-10,0
AFG,Asia,2020-01-11,0
AFG,Asia,2020-01-12,0
AFG,Asia,2020-01-13,0
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$
```

[87]: *# confirm db data*

```
display(Image(filename="screenshots/select_covid_db.png"))
```



```
covid_db=# SELECT * FROM COVID LIMIT 10;
 iso_code | continent |      date      | total_cases
-----+-----+-----+-----
 AFG      | Asia      | 2020-01-05     |          0
 AFG      | Asia      | 2020-01-06     |          0
 AFG      | Asia      | 2020-01-07     |          0
 AFG      | Asia      | 2020-01-08     |          0
 AFG      | Asia      | 2020-01-09     |          0
 AFG      | Asia      | 2020-01-10     |          0
 AFG      | Asia      | 2020-01-11     |          0
 AFG      | Asia      | 2020-01-12     |          0
 AFG      | Asia      | 2020-01-13     |          0
 AFG      | Asia      | 2020-01-14     |          0
(10 rows)

covid_db=#
```

1.11 11. Task 8: Automating the ETL Process with Cron

1.11.1 Step 1: Open the Crontab Editor

To schedule the ETL script to run daily at **10 minutes past midnight (00:10 AM)**, open the crontab editor by running:

```
crontab -e
```

1.11.2 Step 2: Add the Cron Job

Append the following line to schedule the script execution:

```
10 0 * * * /path/to/ETL_script.sh >> /path/to/etl_log.txt 2>&1
```

Explanation:

- 10 0 * * * → Runs the script at **12:10 AM** every day.
- /path/to/ETL_script.sh → path to my ETL script.
- >> /path/to/etl_log.txt 2>&1 → Redirects both **output and errors** to the log file for debugging.

1.11.3 Step 3: Verify the Cron Job

To confirm that the cron job has been added, run:

```
crontab -l
```

This should display the scheduled job.

1.11.4 Step 4: Test the Cron Job

To test whether the scheduled task runs correctly, I manually execute:

```
run-parts --test /etc/cron.daily
```

```
[88]: # confirm cron tab
display(Image(filename="screenshots/crontab.png"))
```

```
simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_management_for_data_science
simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_m... x  simpcy@simpcy: ~/Documents/Development/PACE/data_acquisition_and_m... x
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
10 0 * * * /bin/bash /home/simpcy/Documents/Development/PACE/data_acquisition_and_management_for_data_science/ETL_script
.sh >> /home/simpcy/Documents/Development/PACE/data_acquisition_and_management_for_data_science/ETL_log.txt 2>&1
(base) simpcy@simpcy:~/Documents/Development/PACE/data_acquisition_and_management_for_data_science$
```

1.12 Conclusion

This assignment successfully demonstrated the implementation of an **ETL (Extract, Transform, Load) pipeline** using Bash scripting and PostgreSQL. The process involved:

- Extracting COVID-19 data from an external source using `curl`.
- Transforming the dataset by selecting specific columns and ensuring data integrity.
- Loading the transformed data into a PostgreSQL database with an automated table creation mechanism.
- Automating the entire ETL workflow using `cron` to execute the script daily at 12:10 AM.

The project highlights key **data engineering principles**, including **automation, logging, and structured database management**. By leveraging Bash scripting and Python, we ensured an efficient and scalable solution for handling real-world data ingestion and storage.

This ETL pipeline serves as a foundational framework that can be extended to accommodate more complex data transformations and analytics in the future.

1.12.1 Future Enhancements

To further improve the pipeline, the following enhancements could be considered: - **Data Validation**: Implementing stricter data integrity checks before loading into the database. - **Error Handling**: Enhancing exception handling in both Bash and Python scripts to recover from failures. - **Monitoring & Alerts**: Setting up alerts (e.g., email or Slack notifications) in case of ETL failures. - **Data Archiving**: Storing historical versions of the dataset to track changes over time.

By implementing these improvements, the ETL pipeline can be made even more robust, ensuring **higher reliability and scalability** for larger datasets and more frequent data updates. Now that

the script handles database creation, table setup, and data insertion, the next task is to ensure proper execution and automation using `cron`.